

MockStream

Product Requirements Document (PRD) & Technical Stack Architecture Specification

1. Product Requirements Document (PRD)

MockStream addresses a critical challenge in modern API development: the friction of handling third-party webhooks inside secure, local sandbox networks. Below is the structured matrix detailing core feature sets and business-critical constraints required for implementation.

Feature Component	Requirement Category	Functional Specification & Bounds
Persistent Public Endpoints	Core Functionality	Provides dedicated, static public routing targets. Eliminates the necessity of modifying third-party dashboard callback URLs (e.g., Stripe, Shopify) between local development restarts.
Real-Time Inspection Engine	User Experience	A stream-oriented visual terminal designed with an ergonomic dark visual mode. Instantly exposes complete payload diagnostics including comprehensive HTTP Request headers, JSON bodies, and metadata strings.
Request Replay System	Core Functionality	Enables targeted request resubmission directly to localized service hooks (http://localhost:X). Bypasses upstream systems entirely to optimize localized multi-branch edge test logic.
Low-Latency Ingestion Pipeline	System Performance	The edge ingress subsystem must process, acknowledge, and flush outbound HTTP status responses (202 Accepted) within $t \leq 100ms$ to avoid triggering upstream retry delays.
Payload Normalization	Data Reliability	Structured sanitization layer that standardizes conflicting payload shapes into strongly-typed dynamic documents without dropping non-standard header formats.

2. Technical Architecture & Component Selection

To support high-throughput message processing alongside strict state synchronization guarantees, MockStream employs a low-overhead stack optimized for network event orchestration.

Backend & Ingestion Gateway

Node.js / Fastify

Selected for non-blocking asynchronous event loop processing. Fastify delivers optimal performance boundaries under volatile, concurrent connection spikes on ingestion paths.

Streaming Synchronization

WebSockets (ws Node Implementation)

Establishes bidirectional persistent duplex streaming lines to push ingress captures out to visual dashboards instantaneously, minimizing client polling overhead.

Persistence Layer

PostgreSQL (JSONB Storage Profile)

Combines classical ACID transactions with deep semi-structured document agility via indexed JSONB columns to capture heterogeneous payload matrices safely.

Message Broker & Caching

Redis Pub/Sub Architecture

Serves as the central high-velocity transport lane. Decouples the ingestion edge processing loops from the persistent WebSocket connection memory structures.

3. WebSocket Network Contract Specification

Real-time telemetry transfers are governed by structured handshake parameter rules and active validation routines.

Connection Initialization Syntax

```
GET /v1/stream?endpoint_id=ep_8f9a2b&token=ms_live_99217 HTTP/1.1
Host: api.mockstream.dev
Upgrade: websocket
Connection: Upgrade
```

Stream Maintenance Routines

Network connectivity states are validated using periodic low-overhead server pings. The disconnection trigger condition is bound by the calculation:

$$T_{\{drop\}} = T_{\{interval\}} \times 2 + T_{\{tolerance\}}$$

If successive pong signals fail to complete safely within the window bounds, the server breaks the socket and frees active worker allocations immediately to preserve infrastructure resources.